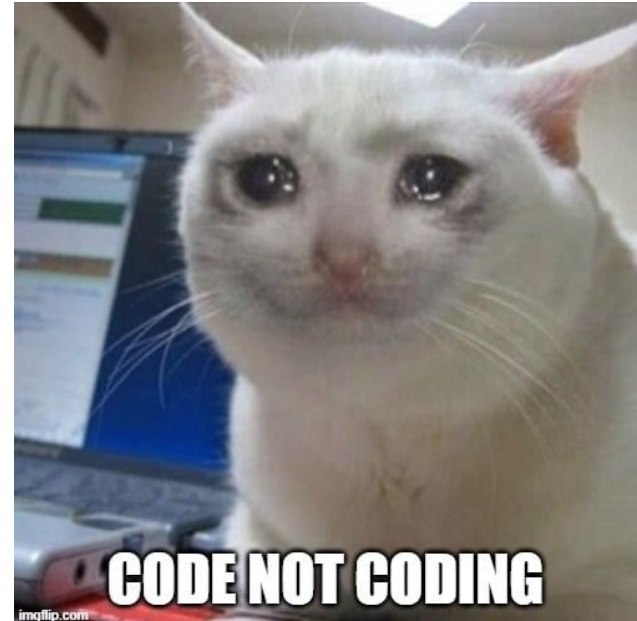


Uoffisiell eksamensforelesning i

TDT4100



Agenda

- 12:15 - 12:30 - Introduksjon og pensumoversikt
- 12:30 - 13:00 - Innkapsling, arv og grensesnitt
- 13:15 - 13:45 - Iteration, lambda, comparison
og funksjonelle grensesnitt
- 13:45 - 14:15 - Streams og filbehandling
- 14:15 - 15:45 - Gjennomgang av en av tidligere eksamener
- 15:45 - 16:00 - Avslutning og spørsmål

Spørsmål underveis

Rekk opp en hånd

Mentimeter



menti.com
3740 5508

Tematisk pensumoversikt:

Prosedyreorientert programmering:

- uttrykk, datatyper og variabler
- primitive datatyper og tabeller
- kontrollstrukturer, funksjonskall og -definisjoner (statiske metoder), og programflyt

Objektorientert programmering

- klasser og instanser, felt, metoder og konstruktører, bruk av **this**
- innkapsling, interaksjon mellom objekter, standardmetoder (navngiving og funksjon) ved innkapsling
- objektoppførsel, grensesnitt og implementasjon av grensesnitt
- diagrammer:
 - objektdiagrammer (objekter med tilstand og koblinger)
 - objekttilstandsdiagrammer
 - sekvensdiagrammer
 - klassediagrammer (klasser og assosiasjoner inkl. kardinalitet)
- testing av objektoppførsel (JUnit og generell testbeskrivelse lik det gjort på forelesning og i øvinger)
- unntakshåndtering
- arv (bruk av **extends** og **super**) og abstrakte klasser (også i kombinasjon med grensesnitt)

Java-teknikker:

- synlighetsmodifikatorene **public**, **private** og **protected**, og **static**-, **abstract**- og **final**-modifikatorene
- bruk av wrapper-klassene (**Integer**, **Boolean**, ...) og **String**-klassen
- **instanceof** og casting
- Collection-rammeverket (**Collection<T>**-, **List<T>**-, **Set<T>**-, **Map<K, V>**-, **Iterator<T>**- og **Iterable<T>**-grensesnittene og **ArrayList<T>**- og **HashMap<K, V>**-klassene), inkludert bruk av **<>** i deklarasjoner, såkalte generics
- sortering med **Comparable<T>** og **Comparator<T>**
- **Exception**-hierarkiet og checked vs. unchecked exceptions
- IO med byte- og tegnstrømmer (**InputStream/OutputStream** og **Reader/Writer** med subclasser) og filer
- Funksjonelle grensesnitt og Java 8-syntaksen for "funksjoner", altså instansiering av funksjonelle grensesnitt.
- Standard funksjonelle grensesnitt, inkl. **Predicate<T>**-, **Consumer<T>**-, **Supplier<T>**-, **Function<T, R>**-, **BiFunction<T1, T2, R>**-, **BinaryOperator<T>** og **UnaryOperator<T>**
- Behandling av data med **Stream**-teknikken (ikke IO, men "objekt-strømmer")
- I tillegg har vi vi introdusert og brukt **Optional** og **Records**. Disse har også dukket opp på eksamen.

Objektorienterte teknikker:

- delegering
- observatør-observert

Innkapsling (encapsulation)

Handler om å gjøre alle variabler private og sette getters og setters for dem

Både metoder og variabler kan være:

- private
- public
- protected

- static
- final

```
public class Person {  
    private String name; // private = restricted access  
  
    // Getter  
    public String getName() {  
        return name;  
    }  
  
    // Setter  
    public void setName(String newName) {  
        this.name = newName;  
    }  
}
```

Arv (inheritance)

Barneklasse extends foreldreklasse

Alle variabler og metoder fra superklassen kan brukes som vanlig i barneklassen

@Override

Lage nye variabler og funksjoner

super()

Polymorphism

Grensesnitt (interface)

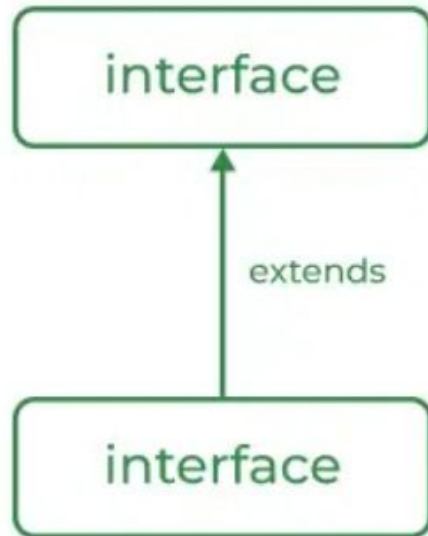
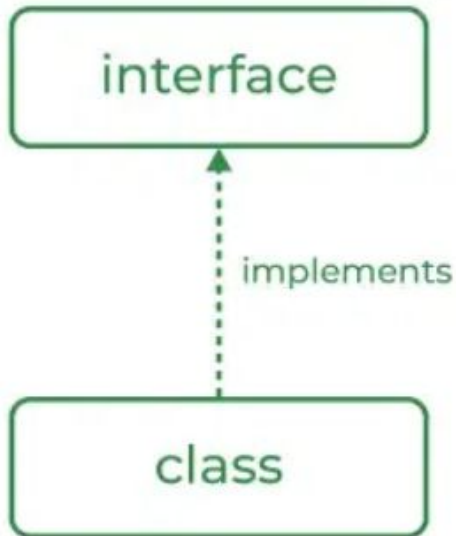
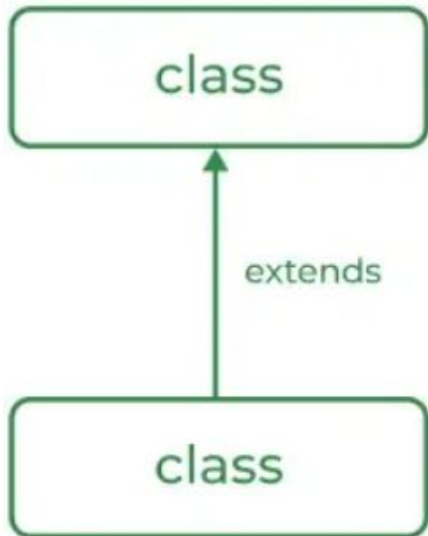
```
interface Grensesnitt { }
```

Klasse implements Grensesnitt

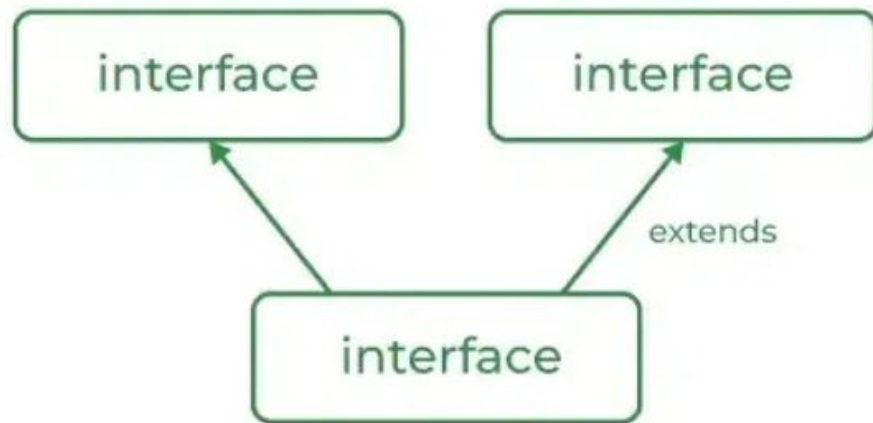
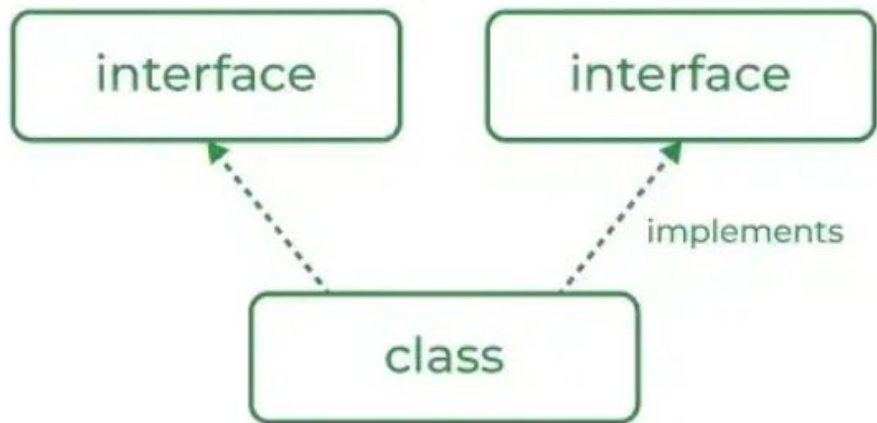
@Override

public, private, static and default methods

Variabler er public, static og final, som default



Multiple inheritance in Java



Repo med eksemplene:

https://github.com/Andrej-52/OOP_kurs/tree/main

Pause

Iteration

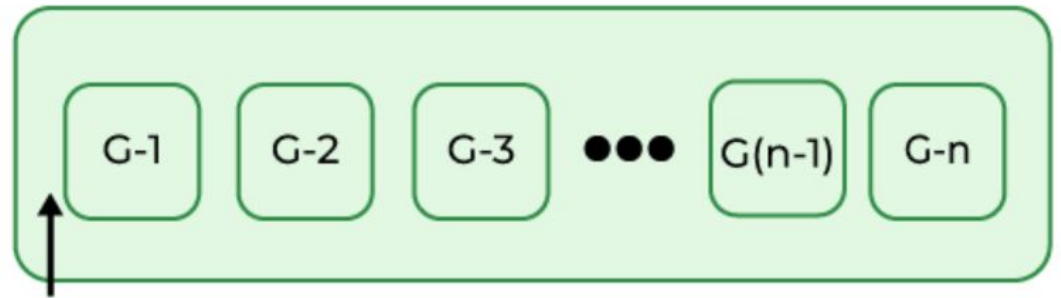
```
Iterator<T> itr = collection.iterator();
```

```
public boolean hasNext();
```

```
public Object next();
```

```
public void remove();
```

```
Iterable<T>?
```



Java Iterator

Comparison - Comparable<T> og Comparator<T>

Comparable - Sortering intærnt i klassen

```
public interface Comparable<T> {  
    int compareTo(T obj);  
}
```

Comparator - Et eksternt objekt som vet hvordan den sorterer to objekter.

```
public int compare(Object obj1, Object obj2):
```

Comparable<T>

```
public interface Comparable<T> {  
    int compareTo(T obj);  
}
```

class Klasse implements Comparable<T>

.sort() bruker compareTo(T obj) funksjonen i seg

Comparator<T>

```
public int compare(Object obj1, Object obj2);
```

Lage egen hjelpeklasse for Comparator

```
public void sort(List list, ComparatorClass c);
```

Metode 2:

```
students.sort((p1, p2) → Integer.compare(p1.age, p2.age));  
students.sort(Comparator.comparing(Student::getName)  
    .thenComparing(Student::getAge));
```

Feature	Comparator	Comparable
Sorting Logic Location	Defined externally	Defined within the class (Internally)
Multiple Sorting Orders	Supported	Not supported
Interface Methods	compare()	compareTo()
Functional Interface	Yes	No
Usage	Flexible and reusable	Simple and tightly coupled

Funksjonelle grensesnitt

Et funksjonelt grensesnitt er et grensesnitt som har nøyaktig én abstrakt metode, og det brukes ofte som mål for lambda-uttrykk i funksjonell programmering, spesielt i Java.

`@FunctionalInterface`

Uendling mange `static` og `default` metoder.

`Comparator<T>`

<u>BiFunction</u>	It accepts two arguments and returns a result.	$R \text{ apply}(T \ t, U \ u)$
<u>UnaryOperator</u>	This is a special case of Function, where input and output types are the same.	$T \text{ apply}(T \ t)$
<u>BinaryOperator</u>	This is a special case of BiFunction, where input and output types are the same.	$T \text{ apply}(T \ t1, T \ t2)$
<u>Consumer</u>	It accepts a single input argument and returns no result.	$\text{void accept}(T \ t)$
<u>Predicate</u>	It accepts a single argument and returns a boolean result.	$\text{boolean test}(T \ t)$
<u>Function</u>	It accepts a single argument and returns a result.	$R \text{ apply}(T \ t)$
<u>Supplier</u>	It does not take any arguments but provides a result.	$T \text{ get}()$

Lambda

Lambda med 0 parametre

```
.getFirst()
```

(Supplier)

Lambda med 1 parameter

```
.filter(string → string ≠ null && !string.isBlank())
```

(Predicate)

Lambda med flere parametre

```
.sorted((a, b) → b.length() - a.length())
```

(BiFunction)

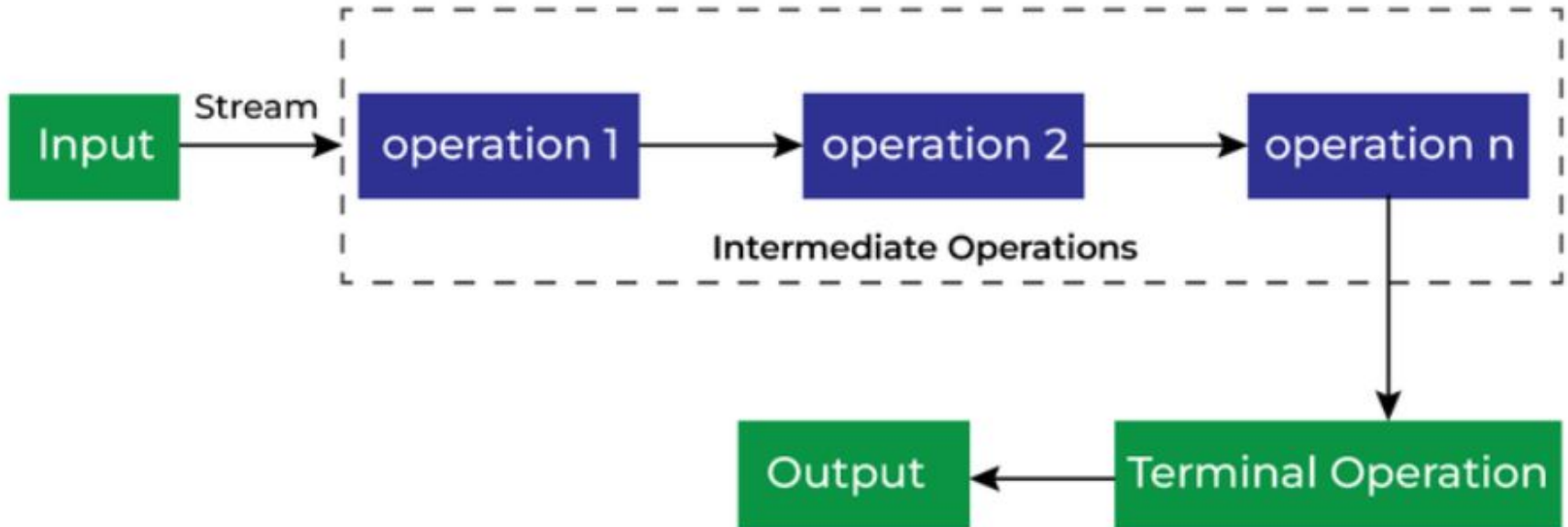
Iterable, Comparator, Predicate

(int arg1, String arg2) → {System.out.println("Two arguments "+arg1+" and "+arg2);}

Argument List Arrow token Body of lambda expression

Streams

Streams er en måte å håndtere en collection steg-for-steg, gjerne med lambdaer.



Streams

- `.map(number → number + 1)`: Gjør en operasjon på hvert element og returnerer det.
- `.map(String::toUpperCase)`: Alternativ for `.map(string → string.toUpperCase())`.
- `.filter(string → string.startsWith("a"))`: Filtrerer på det som stemmer med *predikatet*.
- `.distinct()`: Fjerner alle duplikater.
- `.sorted()`: Sorterer elementene stigende.
- `.min()` og `.max()`: Velger minste og største element
 - `.max((a, b) → a.compareTo(b)).get()`
 - `.max(Integer::compareTo).get()`
- `.anyMatch()`, `.allMatch()` og `.noneMatch()`:
 - `.noneMatch(number → number > 10)`: true om INGEN tall er mer enn 10, ellers false
 - `.anyMatch(string → string.isEmpty())`: true om MINST ÉN streng er tom, ellers false
 - `.allMatch(number → list.contains(number))`: true om list inneholder ALLE tall, ellers false
- `.toList()`: Gjør om resultatet til en `List<T>`. Du kan ikke modifisere listen (legge til/fjerne elementer).
- `.collect(Collectors.toList())`: Gjør også om resultatet til en liste, hvor du kan modifisere den.

Filbehandling

InputStream → Reader

- InputStreamReader
- FileReader

```
var reader = new BufferedReader(new xReader(...))
```

OutputStream → Writer

- outputStream.write(bytes)
- myString.getBytes()

```
var stream = new FileOutputStream("filename.csv");
```

```
public static String[] readStream(InputStream stream) throws IOException {  
    try (BufferedReader reader = new BufferedReader(new InputStreamReader(stream))) {  
        return reader.lines().toArray(String[]::new);  
    }  
}
```

```
public void writeToStream(OutputStream stream, String textToWrite) throws IOException {  
    stream.write(textToWrite.getBytes());  
    stream.flush();  
}
```

Pause

Gjennomgang av tidligere eksamen

Tips og triks

Shortcuts:

- CTRL + F - søk i fil
- CTRL + Shift + F - søk i alle filer (forstørrelsesglasset på venstre i VSC)
- CTRL + P - Søk etter fil
- CTRL + Space - Show autofill (intellisense)
- CTRL + * - Kommenter ut kode

Tips:

- Hold musepekeren over ting
- “Go to definition” (F12, eller CTRL + trykk)
- Ikke bli stuck på én oppgave
- Main metoder (`public static void main(String[] args) {}`)
- [geeksforgeeks.org](https://www.geeksforgeeks.org)

Avsluttende spørsmål

DU PÅ VEI TIL EKSAMEN



ETTER DET KURSET HER